



Sun's 2004 Worldwide Java Developer Conference™

Java™ 2 Platform, Standard Edition (J2SE™) 5.0 new language features, important JSR's, and API's

java.sun.com/javaone/sf

Bruce Brown
Senior Systems Engineer
Sun Microsystems, Inc.



Agenda

Introduction to the “Tiger” Release

Key URLs of Interest

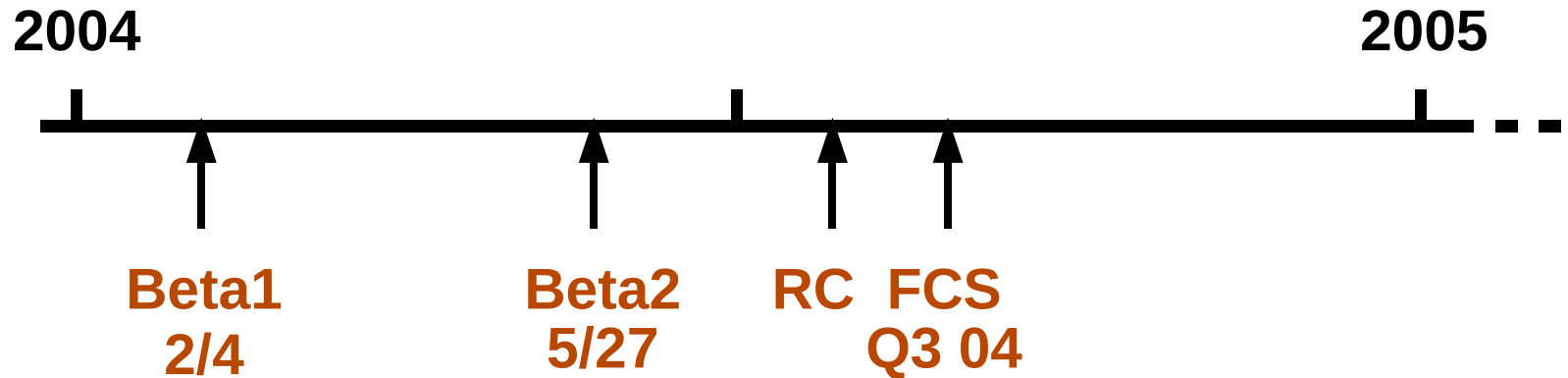
Key New Language Features

Note: Adapted from Calvin Austin's TS-1952 Talk

“Tiger” J2SE 5.0 Technology

- Developed using the Java Community ProcessSM services
- 15 component JSRs for major new features
- Over 160 Experts Worldwide
- Over 100 Features
- Lots of bug fixing and performance work

J2SE 5.0 Technology Timeline



“Tiger” Component JSRs

- 003 JMX™ API
- 013 Decimal Arithmetic
- 014 Generic Types
- 028 SASL
- 114 JDBC™ Rowsets
- 133 New Memory Model
- 160 JMX Remote API
- 163 Profiling API
- 166 Concurrency Utilities
- 174 Monitoring and Management
- 175 Metadata
- 200 Pack transfer format
- 201 Language updates
- 204 Unicode surrogates
- 206 JAXP 1.3

“Tiger” Themes

- Quality
 - Compatibility, compatibility, compatibility!
- Performance and scalability
- Ease of development
- Monitoring and manageability
- Desktop client

Focus on Performance

- Even more support for big heaps
 - Improved concurrent/parallel collectors
- Yet more Java HotSpot™ VM tuning
 - Chipset optimizations
 - AMD64 native compiler
 - Hyper-threading/CMT

More Performance!

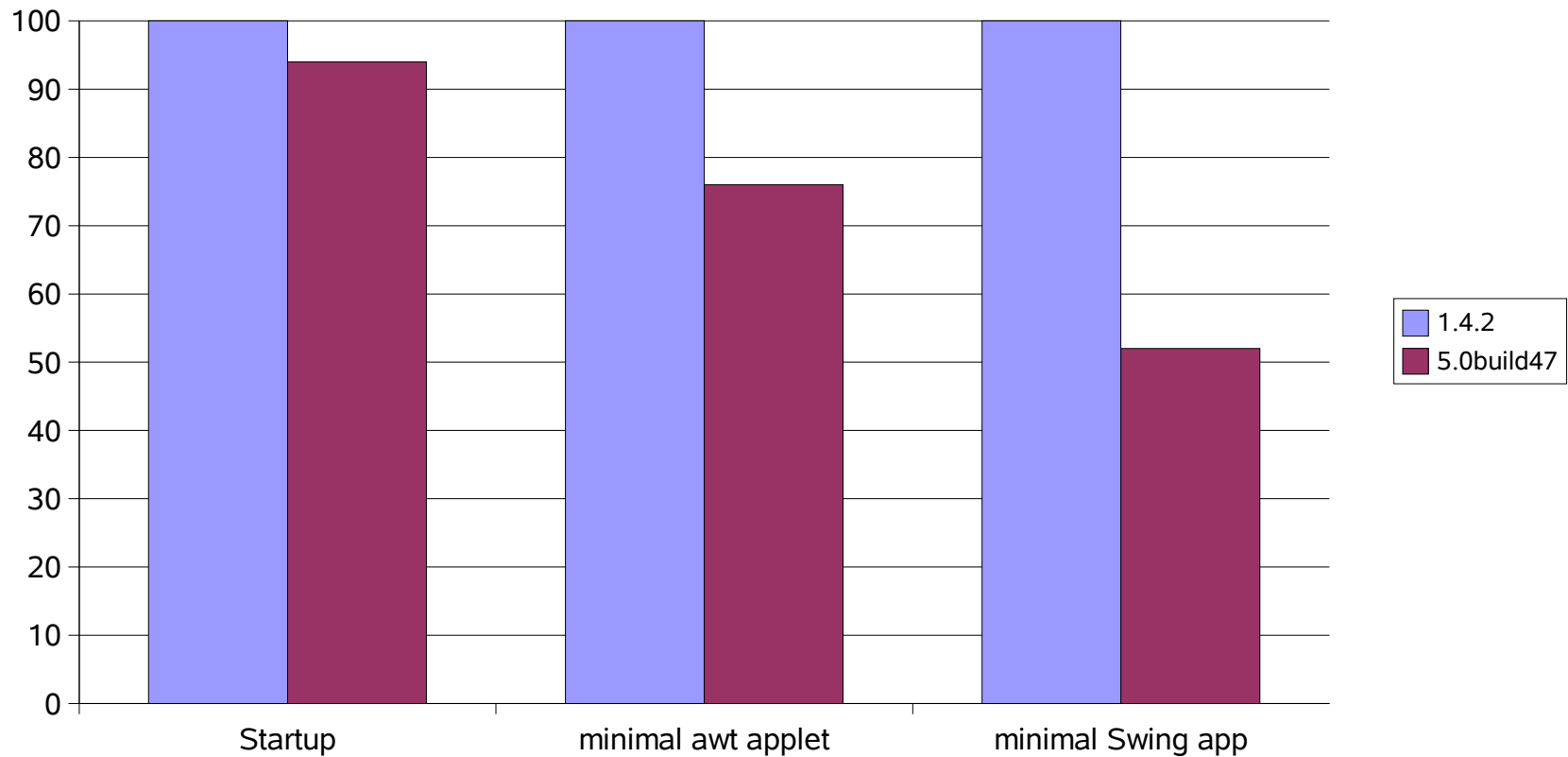
- Faster startup time
 - Especially important for desktop applications
- Class Data Sharing, generated at install time

java -Xshare:off to disable

java -Xshare:dump to re-generate

Internal Tests

Reduction in Windows Startup Time



Performance Ergonomics

- Performance Ergonomics
 - Less reliance on command line flags
 - More support for adaptive self-tuning
- Server class performance for Server class machines out of the box
 - 2 CPU and 2 GB RAM
 - Parallel Garbage Collector
 - Server Java Hotspot VM compiler
 - Max Heap $\frac{1}{4}$ Max Memory or 1GB
 - Initial Heap $\frac{1}{64}$ Max Memory or Max Heap
- Windows 32bit defaults to client VM, all 64 bit installs default to the Server VM.

Key URLs of Interest

- The JavaONE 2004 website:
 - Presentations PDFs
 - Online Labs!
 - And more...
 - <http://javaoneonline.mentorware.net/servlet/mware.servlets.LoginServlet?subsysId=2000>
- What's New Document
 - <http://java.sun.com/j2se/1.5.0/docs/index.html>
 - <http://java.sun.com/j2se/1.5.0/docs/relnotes/features.html>
- <http://jcp.org/en/jsr/detail?id=176>

Ease of Development Changes

- MetaData
- Generics
- Autoboxing
- Enhanced for loops
- Enumerated Types, Static Import
- Covariant Return Types
- Formatted Output/Input
- Varargs
- Concurrency Utilities

You Haven't Tried J2SE 5 Beta 2 Yet?



Compiler Options

- Default javac compiler wants 1.5 compatible source
- Only use -source 1.4 for backward source compatibility
- Only use -source 1.4 -target 1.4 for corner cases

Metadata

- Key new Java programming language idea
- Allows decoration of packages/classes/methods/fields for simpler declarative programming
- Can be retained in class file and also be visible at runtime
- Designed for use by tools, apt tool available in beta2
- Can help catch subtle errors like overloading when you wanted to override - `@Override`
- J2EE JSRs expected to generate new useful annotations.

Java Web Services

Tutorial Example: JAX-RPC Server

Before

```
public interface HelloIF extends Remote {  
    public String sayHello(String s) throws  
        RemoteException;  
}  
  
public class HelloImpl implements HelloIF {  
    public String sayHello(String s) {  
        return "Hello "+s;  
    }  
}
```

After

```
public class HelloImpl {  
    public @remote String HelloImpl(String s) {  
        return "Hello "+s;  
    }  
}
```


APT Tool

- Front end to Java *language* Compiler
- Parses metadata with AnnotationProcessorFactory
- AnnotationProcessorFactory selects an AnnotationProcessor
- **AnnotationProcessor** has a **process()** method to handle metadata tags
- **process()** method has access to a **Filer** class to create source, class or text files

```
PrintWriter pw = env.getFiler().createSourceFile  
("Ex1");
```

Annotations Example

```
1 import java.lang.annotation.*;
```

```
2 @Retention(RetentionPolicy.RUNTIME)
```

```
3 @Target(ElementType.METHOD)
```

```
4 public @interface Test { }
```

Test.java

```
5 public class Foo {
```

```
6     @Test public static void m1() { }
```

```
7     public static void m2() { }
```

```
8     @Test public static void m3() {
```

```
9         throw new RuntimeException("Boom");
```

```
10    }
```

```
11}
```

Foo.java

```
12 import java.lang.reflect.*;
```

```
13 public class RunTests {
```

```
14     public static void main(String[] args) throws Exception {
```

```
15         int passed = 0, failed = 0;
```

```
16         for (Method m : Class.forName(args[0]).getMethods()) {
```

```
17             if (m.isAnnotationPresent(Test.class)) {
```

```
18                 try {
```

```
19                     m.invoke(null);
```

```
20                     passed++;
```

```
21                 } catch (Throwable ex) {
```

```
22                     System.out.printf("Test %s failed: %s %n", m, ex.getCause  
23  
24  
25  
26  
27  
28  
29  
29 }  
    $ java RunTests Foo  
    Test public static void Foo.m3() failed: java.lang.RuntimeException:  
    Boom  
    Passed: 1, Failed 1
```

RunTests.java

Generics

- What happens today?
 - When creating shared algorithms that work on different types, designers have to use the general Object type, e.g., Collections
- What is the problem with that?
 - Developers have to explicitly cast down to retrieve the object they need. This negates type safety checking

Generics Solution

The solution

- Allow the developer to provide a type parameter at declaration time inside `<..>`, e.g., `<Integer>`
- The compiler now can check for type safety
- Collections have been updated to use Generics

Generics Example

Before

```
ArrayList list = new ArrayList();  
list.add(0, new Integer(42));  
int total = ((Integer)list.get(0)).intValue();
```

After

```
ArrayList<Integer> list = new ArrayList<Integer>();  
list.add(0, new Integer(42));  
int total = list.get(0).intValue();
```

Writing Generics APIs

- Warning: generifying old APIs needs careful thought
 - See <http://java.sun.com/j2se/1.5/pdf/generics-tutorial.pdf>
- Look at Collections API source for examples
- For portability, old Collections API is compatible with Generic Collections API

Pre-Generic API

```
public class MoneyBox {
    private Vector pot =new Vector();

    public void addCoin(Object o) {
        pot.add(o);
    }

    public Object getMoney() {
        return pot.firstElement();
    }

    public static void main(String[] args) {
        MoneyBox m= new MoneyBox();
        m.addCoin("iou");
        m.addCoin(new Integer(3));
        System.out.println(((Integer)m.getMoney()).
intValue());
    }
}
```

Pre-Generic Result

```
$ javac MoneyBox.java
```

```
$ java MoneyBox
```

```
Exception in thread "main"  
    java.lang.ClassCastException:  
        java.lang.String  
at MoneyBox.main(MoneyBox.java:18)
```


Generified Version

```
public class MoneyBox<M> {
    Vector<M> pot =new Vector<M>();

    public void addCoin(M o) {
        pot.add(o);
    }

    public M getMoney() {
        return pot.firstElement();
    }

    public static void main(String[] args) {
        MoneyBox<Integer> m= new MoneyBox<Integer>
        ();
        m.addCoin("iou");
        m.addCoin(new Integer(3));
        System.out.println(m.getMoney());
    }
}
```

Generic Result

```
$ javac MoneyBox.java
```

```
MoneyBox.java:15: addCoin  
    (java.lang.Integer) in  
    MoneyBox<java.lang.Integer> cannot be  
    applied to (java.lang.String)
```

```
m.addCoin("iou");
```

^

1 error

Autoboxing of Primitive Types

- Conversion to and from primitive types to their Object equivalent

Before

```
list.add(0, new Integer(42));  
int total = (list.get(0)).intValue();
```

After

```
list.add(0, 42);  
int total = list.get(0);
```

Enumerated Types

- Safer than emulated enumerations with constants – not typesafe!
- Printed values are uninformative

```
public enum StopLight { red, amber, green }
```

```
StopLight t=StopLight.red;  
StopLight t=1;//error  
StopLight t=StopLight.blue; //error
```

Enum Types Example

```
1 public enum Planet {
2     MERCURY (3.303e+23, 2.4397e6), //Params to Constructor
3     VENUS   (4.869e+24, 6.0518e6),
4     EARTH   (5.976e+24, 6.37814e6),
5     MARS    (6.421e+23, 3.3972e6),
6     JUPITER (1.9e+27, 7.1492e7),
7     SATURN  (5.688e+26, 6.0268e7),
8     URANUS  (8.686e+25, 2.5559e7),
9     NEPTUNE (1.024e+26, 2.4746e7),
10    PLUTO   (1.27e+22, 1.137e6);
11    private final double mass;    // in kilograms
12    private final double radius; // in meters
13    Planet(double mass, double radius) {
14        this.mass = mass;
15        this.radius = radius;
16    }
17    private double mass() { return mass; }
18    private double radius() { return radius; }
19    // universal gravitational constant (m3 kg-1 s-2)
20    public static final double G = 6.67300E-11;
21    double surfaceGravity() {
22        return G * mass / (radius * radius);
23    }
24    double surfaceWeight(double otherMass) {
25        return otherMass * surfaceGravity();
26    }
27 }
```

```
public static void main(String[] args) {
    double earthWeight = Double.parseDouble(args[0]);
    double mass = earthWeight/EARTH.surfaceGravity();
    for (Planet p : Planet.values())
        System.out.printf("Your weight on %s is %f%n",
                           p, p.surfaceWeight(mass));
}
```

\$ java Planet 175

Your weight on MERCURY is 66.107583

Your weight on VENUS is 158.374842

Your weight on EARTH is 175.000000

Your weight on MARS is 66.279007

Your weight on JUPITER is 442.847567

Your weight on SATURN is 186.552719

Your weight on URANUS is 158.397260

Your weight on NEPTUNE is 199.207413

Your weight on PLUTO is 11.703031

Co-variant Return Types

1.4 Version

An argument or return type in an overridden method that is made more specialized is called to be covariant.

```
public class MyDialog {
    MyDialog getDialog() {
        return this;
    }
}

public class MyFileDialog extends MyDialog{
    public boolean isDirectory() {

        MyDialog getDialog() {
            return this;
        }
    }

    ...

    MyFileDialog f = new MyFileDialog();

    MyFileDialog copy = (MyFileDialog) f.getDialog();
}
```

Co-variant Return Types (2)

5.0 Version

```
class MyDialog {  
    MyDialog getDialog() {  
        return this;  
    }  
}  
  
class MyFileDialog extends MyDialog{  
    public boolean isDirectory() {  
    }  
    // was MyDialog getDialog() {  
    MyFileDialog getDialog() {  
        return this;  
    }  
}  
  
MyFileDialog f = new MyFileDialog();  
MyFileDialog copy = f.getDialog();
```

“The type of the return is allowed to vary in the same direction that you subclass”

Varargs

- Three periods after the final parameter's type indicate that the final argument may be passed as an array or as a sequence of arguments.
- Synergy with reflection and printf

```
void argtest(String ... args) {  
    for (int i=0; i < args.length; i++) {  
    }  
}  
  
//  
argtest("test", "data");
```


Formatted Output

- printf functionality is added to J2SE 5.0 technology
 - Compatible with C version
 - Helps migrate legacy code
- You can do things like:

```
out.printf("name count\n");  
out.printf("%s %5d\n", user, total);
```
- Note: Use %n instead of \n for platform-independent newline

Formatted Input

- Scanner API provides simplified Input functionality

C version

```
scanf ("%s", name) ;  
scanf ("%d", age) ;
```

Java Version

```
Scanner s=new Scanner(System.in) ;  
String name= s.next() ;  
int age=s.nextInt() ;  
s.close() ;
```

Concurrency Utilities

- Concurrency Utility Library (JSR-166)
 - Being led by Doug Lea
 - Does for concurrent designs what Collections did for data structures!
- Task Scheduling Framework
- Concurrent Collections
 - Queue and Blocking Queue
- Atomic Variables
 - Primitive types or references, compare and set methods
- Nanosecond-granularity timing

Concurrency Utilities

- Locks
 - Goes beyond synchronization keyword by providing timeout to acquire, multiple condition variables and more...
- Synchronizers
 - Semaphores – counting semaphore, conceptually a set of permits
 - Barriers – a set of threads wait to reach a common point
 - Latches – thread(s) wait for other thread(s) to complete operations
 - Exchangers – sync point where 2 thread exchange objects

Summary

- This was a fast-track overview
 - There are over 100 features
- Watch out for “Tiger”!
 - Performance, ease of development, RAS and Desktop Client



Sun's 2004 Worldwide Java Developer Conference™

Java™ 2 Platform, Standard Edition (J2SE™) 5.0 new language features, important JSR's, and API's

java.sun.com/javaone/sf

Bruce Brown
Senior Systems Engineer
Sun Microsystems, Inc.

